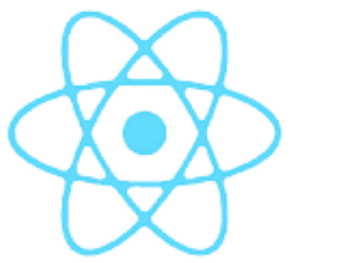
QUESTION BANK

ADVANCED

WEB DESIGNING

TYBCA SEM 5



TYBCA - SEM 5

MongoDB

1. **What are the advantages of NoSQL databases over traditional relational databases?**

NoSQL databases offer a number of **advantages** over traditional relational databases, including:

- **Scalability**: NoSQL databases can be scaled horizontally, which means that you can add more servers to the database as needed to handle more data. This is in contrast to relational databases, which are typically scaled vertically, which means that you need to upgrade to more powerful servers as your data grows.
- **Flexibility**: NoSQL databases do not require a rigid schema, which means that you can store data in a variety of formats. This makes them ideal for storing unstructured or semi-structured data, such as social media data, sensor data, and log files.
- **Performance**: NoSQL databases can be faster than relational databases for certain types of queries, such as queries that involve joins. This is because NoSQL databases are typically optimized for specific types of data and queries.
- **Cost-effectiveness**: NoSQL databases can be more cost-effective than relational databases for large-scale applications. This is because they can be scaled horizontally, which means that you can add more servers as needed without having to upgrade to more expensive hardware.

However, NoSQL databases also have some **disadvantages**, such as:

- **Data consistency**: NoSQL databases do not always guarantee data consistency. This means that there is a possibility that two users may read different versions of the same data.
- **Limited ACID compliance**: ACID stands for Atomicity, Consistency, Isolation, and Durability. NoSQL databases do not always guarantee all of these properties.
- **Lack of standardization**: There is no single standard for NoSQL databases. This can make it difficult to choose a database and to migrate data between databases.

Overall, NoSQL databases offer a number of advantages over traditional relational databases. However, it is important to choose the right database for your specific needs. If you need to store large amounts of unstructured or semi-structured data, or if you need to scale your database horizontally,

then a NoSQL database may be a good choice. However, if you need to guarantee data consistency or if you need to comply with ACID requirements, then a relational database may be a better choice.

2. List the data types supported by MongoDB.

Data Type	Description	Examples
String	Stores text data.	"Hello, World!", "John Doe"
Integer	Represents whole numbers.	42, -10, 1000
Double	Stores floating-point numbers.	3.14, -0.001, 2.718
Boolean	Represents true or false values.	true, false, 1, 0
Date	Stores date and time information.	ISODate("2023-09-09T12:00:00Z")
ObjectId	A unique identifier for documents.	ObjectId("5f5b4c1f1c11a22d4c062bca")
Array	An ordered list of values or elements.	[1, 2, 3], ["apple", "banana", "cherry"]
Embedded Document	Nested documents within a document.	{"name": "John", "age": 30}
Null	Represents a missing or undefined value.	null, undefined
Binary Data	Stores binary data, like images or files.	Binary data (e.g., image in bytes)
Regular Expression	Stores regular expression patterns.	/pattern/i, /abc[0-9]/
Geospatial	Supports geographical data and queries.	{"type": "Point", "coordinates": [1.23, 4.56]}
Decimal128	High-precision decimal numbers.	Decimal128("123.45678901234567890123456789")

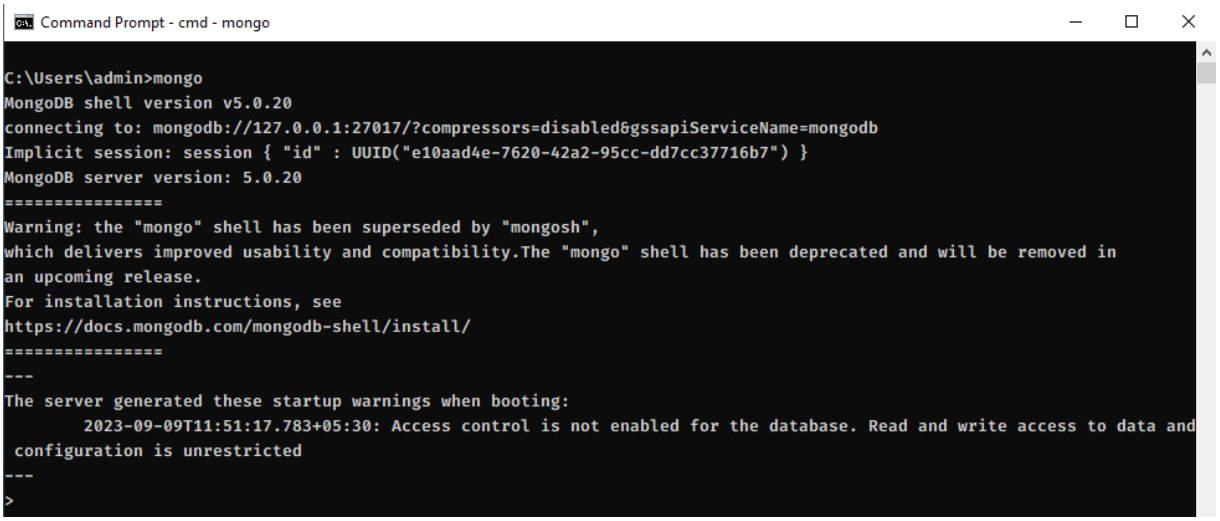
3. How do you create a database and a collection in MongoDB?

In MongoDB, you can create a database and a collection using the MongoDB shell or a MongoDB driver in your preferred programming language. Here are the steps to create a database and a collection with examples:

Step 1: Connect to MongoDB

Before creating a database and a collection, you need to connect to your MongoDB server. You can do this using the MongoDB shell or a MongoDB driver. For this example, let's use the MongoDB shell:

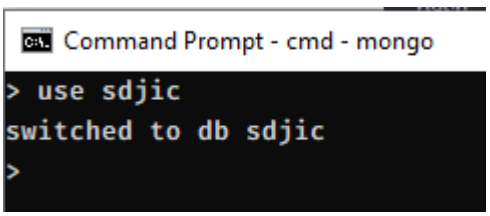
$> mongo



Note: Make sure you installed correct version of MongoDB and environment variable “Path” set to the installed mongodb directory.

Step 2: Create a Database

To create a database, you can use the `**use**` command followed by the desired database name. MongoDB will create the database if it doesn't already exist. For example, to create a database named "**sdjic**," you can do:



Step 3: Create a Collection

Once you've selected or created a database, you can create a collection within that database. Use the db.createCollection() method to create a collection. For example, let's create a collection named "**students**" within the "**sdjic**" database:

```
db.createCollection("students")
```

Step 4: Insert an Example Student with an Indian Name

You can insert a student document with an Indian name into the "students" collection like this:

```
db.students.insert({
    name: "Nehal Patel",
    age: 25,
    gender: "Male",
    nationality: "Indian",
    subjects: ["Computer Science", "Mathematics", "Physics"],
    address: {
        street: "123 Vesu Road",
        city: "Surat",
        state: "Gujarat",
        country: "India"
    }
})
```

This command inserts a student document with fields such as "name," "age," "gender," "nationality," "subjects," and "address" into the "students" collection. The name "Rajesh Patel" is an example of an Indian name.

Now, you have created the "sdjic" database, the "students" collection, and inserted an example student with an Indian name into MongoDB. You can continue to insert more student records or perform various queries and operations on this collection as needed.

4. What are the CRUD operations in MongoDB?

In MongoDB, CRUD operations refer to the basic operations you can perform on data stored in a MongoDB database. CRUD stands for Create, Read, Update, and Delete. Here's how these operations correspond to MongoDB commands using the previous example of a student document:

Create (C):

Insert is used to create new documents in a collection.

```
db.students.insert({
    name: "Nehal Patel",
    age: 25,
    gender: "Male",
    nationality: "Indian",
    subjects: ["Computer Science", "Mathematics", "Physics"],
    address: {
        street: "123 Vesu Road",
        city: "Surat",
        state: "Gujarat",
        country: "India"
    }
})
```

Read (R):

Find is used to retrieve documents from a collection.

```
db.students.find({ name: "Nehal Patel" })
```

Update (U):

UpdateOne or **UpdateMany** is used to modify existing documents in a collection.

Example (updating "Devashya Patel" to "Nehal Patel" and changing the address):

```
db.students.updateOne(
    { name: "Devashya Patel" }, // Specify the existing student document
    {
        $set: {
            name: "Nehal Patel", // Update the name to "Nehal Patel"
            "address.city": "Surat", // Update the city to "Surat"
            "address.street": "Vesu" // Update the street to "Vesu"
        }
    }
)
```

Delete (D):

DeleteOne or **DeleteMany** is used to remove documents from a collection.

Example (deleting a student with a specific name):

```
db.students.deleteOne({ name: "Nehal Patel" })
```

5. Explain the use of the following operators in MongoDB:

a. Projection

b. Update

c. Limit()

d. Sort()

a. Projection:

Use: Projection allows you to control which fields are returned in the query result. You can specify which fields to include or exclude from the documents in the result.

Example: Suppose you have a collection of students, and you want to retrieve only their names and ages.

```
// Retrieve only the "name" and "age" fields from the documents
db.students.find({}, { name: 1, age: 1, _id: 0 })
```

This query will return documents with only the "name" and "age" fields.

b. Update:

Use: The update operation allows you to modify existing documents in a collection. You can update specific fields or replace entire documents.

Example: Let's say you want to update the age of a student named "Nehal Patel" to 26 years old.

```
// Update the age of "Nehal Patel" to 26
db.students.updateOne({ name: "Nehal Patel" }, { $set: { age: 26 } })
```

This query will find the document with the name "Nehal Patel" and update the age field to 26.

c. Limit():

Use: The **limit()** method is used to restrict the number of documents returned in a query result. It limits the result set to a specified number of documents.

Example: If you want to retrieve only the first two students in your collection:

```
// Retrieve the first two students
db.students.find().limit(2)
```

This query will return only the first two documents from the collection.

d. Sort():

Use: The **sort()** method is used to sort the documents in the result set based on one or more fields. You can specify the sorting order (ascending or descending) for each field.

Example: If you want to retrieve students sorted by their ages in descending order:

```
// Retrieve students sorted by age in descending order
db.students.find().sort({ age: -1 })
```

This query will return students sorted by their ages in descending order, so the oldest students will appear first in the result.

These operators provide powerful capabilities for querying and manipulating data in MongoDB, allowing you to tailor your queries to your specific requirements.

6. **What are the limitations of MongoDB?**

MongoDB is a popular NoSQL database system, but like any technology, it has its limitations. Here are some of the common limitations of MongoDB:

1. No ACID Transactions Across Multiple Documents: MongoDB supports ACID transactions at the document level, but transactions that involve multiple documents across different collections or databases, are not supported in all versions (though it's available in some newer versions). This can be a limitation for complex transactions.

2. Memory Usage: MongoDB's performance is highly dependent on available memory. If your dataset exceeds the available RAM, it may lead to performance issues and increased I/O operations.

3. Limited Joins: MongoDB is designed for schema flexibility, which means it doesn't support traditional SQL joins. While you can achieve similar results with embedded documents or using the `$lookup` aggregation stage, it's not as straightforward as SQL joins.

4. Lack of Transactions for Some Data Models: While MongoDB supports multi-document transactions, some data models, such as many-to-many relationships, can be challenging to implement with transactions.

5. Storage Space: MongoDB may require more storage space compared to relational databases because it stores field names in each document. This can lead to increased storage costs.

6. Indexing Complexity: While MongoDB supports indexing, you need to carefully plan and manage indexes to ensure optimal query performance. Inadequate or excessive indexing can impact performance.

7. No Built-in Join Optimization: MongoDB doesn't have a query optimizer that can automatically optimize queries. Developers need to manually analyze and optimize queries.

8. Limited Analytics Capabilities: MongoDB is primarily designed for operational data, and its analytics capabilities are not as robust as dedicated data warehousing solutions.

9. Geospatial Limitations: While MongoDB has geospatial capabilities, it may not be as feature-rich as dedicated geospatial databases for complex geospatial queries.

10. Data Size Limitation: Some older versions of MongoDB had a 16 MB document size limit, which could be a limitation for large documents. Newer versions have increased this limit, but it's still something to consider.

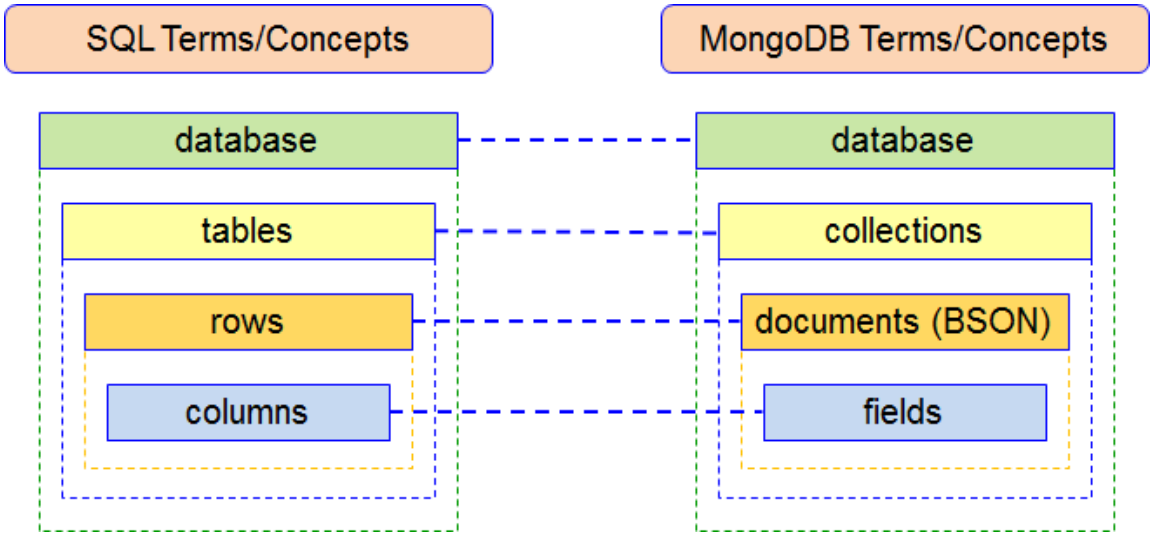
11. Complexity of Schema Design: MongoDB's flexibility can lead to complex schema designs, which can be challenging to manage and maintain.

12. Security: MongoDB's default configuration may not be secure, and securing it properly requires careful setup and management.

It's important to note that MongoDB continues to evolve, and many of these limitations have been addressed or improved in newer versions. When using MongoDB, it's essential to carefully evaluate your use case and consider whether the database's features and limitations align with your requirements. Additionally, some limitations can be mitigated through proper design, optimization, and configuration.

7. Compare MongoDB with RDBMS with examples.

Comparison between MongoDB (a NoSQL database) and a traditional Relational Database Management System (RDBMS) like MySQL



Aspect	MongoDB (NoSQL)	RDBMS (e.g., MySQL)
Data Model	Document-Oriented (JSON-like BSON)	Table-Based (Rows and Columns)
Schema	Dynamic (Flexible Schema)	Static (Fixed Schema)

Examples of Data Model	JSON-like Documents	Tables with Rows and Columns
Example Document	**{ "name": "John", "age": 30 }**	**ID: 1, Name: "John", Age: 30**
Scalability	Horizontal Scalability (Sharding)	Vertical Scalability (Adding More Resources)
Joins	No Built-in Joins (Denormalized Data)	Supports Complex Joins
ACID Transactions	Supports Single-Document Transactions	Supports Multi-Table ACID Transactions
Example of Transaction	**db.collection.updateOne()**	**START TRANSACTION; ... COMMIT;**
Query Language	Rich Query Language (MongoDB Query)	SQL (Structured Query Language)
Example of Query	**db.collection.find({ age: 30 })**	**SELECT * FROM users WHERE age = 30;**
Schema Changes	Easily Handles Schema Evolution	Schema Changes May Require Downtime
Atomicity of Writes	Supports Atomic Writes at Document Level	Supports Atomicity at Transaction Level
Example of Index	**db.collection.createIndex({ field: 1 })**	**CREATE INDEX index_name ON table_name (column_name);**

Please note that the choice between MongoDB (NoSQL) and an RDBMS depends on the specific requirements of your application. MongoDB is well-suited for scenarios with flexible data structures and scalability needs, while RDBMS systems excel in handling structured data with complex relationships and support for ACID transactions.

MongoDB MCQs

1. What type of database is MongoDB?

a. Relational Database

b. NoSQL Database *

c. Graph Database

d. Key-Value Database

2. Which of the following represents a single unit of data in MongoDB?

a. Table

b. Row

c. Document *

d. Column

3. What is the primary key in a MongoDB document called?

a. Row ID

b. Document ID

c. Object ID *

d. Primary Key

4. Which query language is used for querying MongoDB documents?

a. SQL

b. NoSQL Query Language

c. MongoDB Query Language*

d. JSON Query Language

5. In MongoDB, what is the purpose of an index?

a. To define the data structure

b. To store large binary data

c. To speed up data retrieval and querying *

d. To establish relationships between collections

6. What is the default port for the MongoDB server?

a. 3306

b. 27017 *

c. 1433

d. 8080

7. Which MongoDB aggregation stage is used for filtering documents in a collection?

a. $project

b. $match *

c. $group

d. $sort

8. What is the maximum size of a single BSON document in MongoDB?

a. 16 MB *

b. 64 KB

c. 1 GB

d. 256 MB

9. What does "ACID" stand for in the context of MongoDB?

a. Atomic, Consistent, Isolated, Durable

b. Aggregated, Concurrent, Indexed, Distributed

c. Asynchronous, Continuous, Incremental, Decentralized

d. MongoDB does not support ACID transactions *

10. What does BSON stand for?

a. Binary Structured Object Notation

b. Binary Serialized Object Notation

c. Binary Simple Object Notation

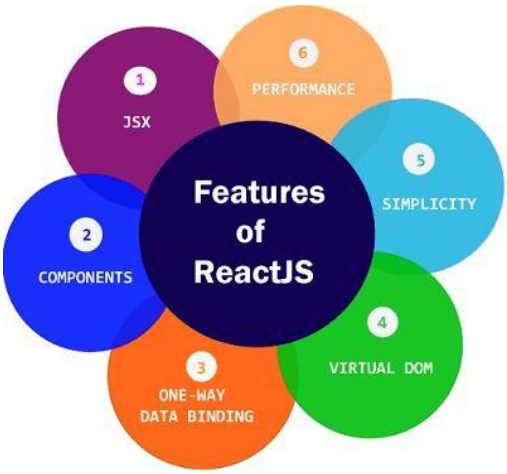
d. Binary JSON (JavaScript Object Notation) *

TYBCA- SEM 5

React.js

Unit 2: Fundamentals of React.js

1. **Explain the key concepts of React.js. What makes React different from other JavaScript libraries or frameworks?**



React.js is a popular JavaScript library used for building user interfaces (UI) in web applications. Here are the key concepts that make React different from other JavaScript libraries or frameworks:

1. Component-Based: React organizes the UI into reusable building blocks called "components." These components represent different parts of a web page, like buttons, forms, or entire sections. For example, you can have a "Header" component and a "Footer" component.

2. Virtual DOM: React uses a virtual representation of the actual DOM (Document Object Model). When changes occur in your application, React first updates this virtual DOM, which is faster than directly manipulating the real DOM. It then calculates the most efficient way to update the actual DOM, improving performance.

3. One-Way Data Flow: React enforces a unidirectional data flow. This means that data flows in one direction, typically from a parent component to its child components. For instance, a parent component can pass data to its child component through props.

4. Reusable Components: React components are highly reusable. You can use the same component in different parts of your application, reducing code duplication and making maintenance easier.

5. Declarative Syntax: React uses a declarative approach to define how the UI should look based on the current application state. Instead of manually

changing the DOM to reflect changes, you describe what you want the UI to look like, and React takes care of updating it.

In summary, React.js simplifies building web applications by breaking them down into reusable components, optimizing performance with a virtual DOM, enforcing a one-way data flow, promoting component reusability, and using a declarative syntax to define UI behavior. These concepts make React a powerful and efficient choice for building user interfaces.

2. **How can you integrate React with HTML? Provide an example of rendering a React component within an HTML file.**
   Or
   Create a new project in for React.js and create sample component

- **Create React Project using NPM `create-react-app`**

```
C:\Users\admin\Desktop\tybca>npx create-react-app sdjic
```

The `npx create-react-app` command is a way to create a new React application. It installs the necessary dependencies, such as React and ReactDOM.

The `sdjic` part of the command is the name of the new project. You can choose any name you want.

The `npx create-react-app sdjic` command will create the following folder and files:

```
sdjic/
├── README.md
├── package.json
├── public/
│   ├── favicon.ico
│   └── index.html
└── src/
    ├── App.js
    └── index.js
```

- The README.md file is a Markdown file that contains information about the project, such as its name, description, and version number.
- The package.json file is a file that lists the dependencies of the project. These are the packages that are needed to run the project.
- The public folder contains the static files of the project. These are the files that are not changed by the code, such as the HTML file and the favicon.
- The public/index.html contain the basic HTML structure of the project. This file contain a div with root id,
- The src folder contains the source code of the project. This is where you will write your React components.
- The App.js file is the main component of the project. This is the component that is rendered when the project starts.
- The index.js file is the entry point of the project. This is the file that is executed when the project starts.

src/index.js

```
1  import React from 'react';
2  import ReactDOM from 'react-dom/client';
3  import App from './App';
4
5  const root = ReactDOM.createRoot(document.getElementById('root'));
6  root.render(
7    <React.StrictMode>
8      <App />
9    </React.StrictMode>
10 );
```

src/app.js

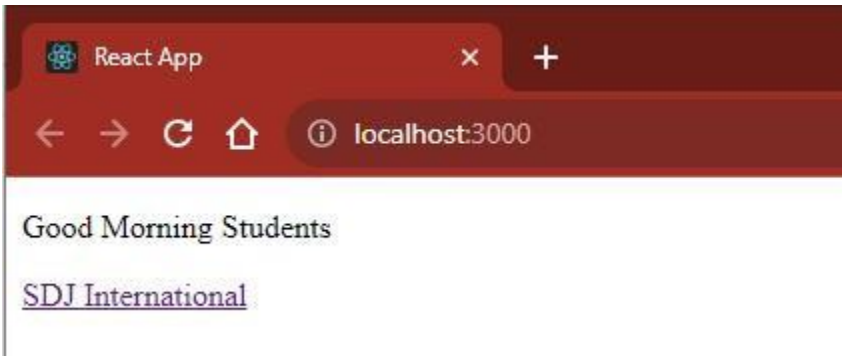
```
function App() {
  return (
    <div>
      <p>Good Morning Students</p>
      <a href="https://www.sdjic.org" target="_blank">
        SDJ International
      </a>
    </div>
  );
}

export default App;

```

To run the project write following code.

```
C:\Users\admin\Desktop\tybca\sdjic>npm run start
```



Here **App** component is a functional component where it is using JSX code which will display simple message with a Link.

3. **In React, what is meant by "Components within components"? Give an example of nesting components.**

In React, "Components within components" refers to the practice of nesting one or more React components inside another component. This is a fundamental concept in React that allows you to build complex user interfaces by composing smaller, reusable components.

Let's illustrate this concept with an example using functional components:

Suppose you're building a simple blog post component that consists of a post title, content, and comments. You can break this down into two smaller components: Post, Comments. Here's how you can nest components within one another:

```
// Post.js - Parent Component
import React from 'react';
import Comment from './Comment';

function Post() {
    return (
        <div className="post">
            <h1 className="title">React Blog Post</h1>
            <p className="content">
                This is the content of the blog post.
            </p>
            <Comment />
        </div>
    );
}

export default Post;
```

```jsx
// Comment.js - Child Component
import React from 'react';

function Comment() {
    return (
        <div className="comments">
            <h3>Comments</h3>
            <ul>
                <li>Comment 1</li>
                <li>Comment 2</li>
                <li>Comment 3</li>
            </ul>
        </div>
    );
}

export default Comment;
```

In this example, the Comment component handles the rendering of the comments section within the Post component. This approach still demonstrates the concept of "Components within components" by nesting the Comment component inside the Post component, creating a hierarchical structure in your React application.

4. **What is the purpose of passing data through Props in React? Provide a practical scenario where you would use Props to pass data between components.**

In React, passing data through **Props** (short for "properties") serves the essential purpose of enabling communication and data flow between parent and child components. Props allow you to send data from a parent component to a child component, making it possible for child components to receive and render data dynamically based on the parent's state or data.

The primary purposes of passing data through Props in React are:

Data Sharing: Props provide a way to share data between components, allowing you to pass information from a parent component down to its child components. This is crucial for building reusable and composable components.

Component Customization: Props allow you to customize the behavior and appearance of child components based on the data provided by the parent component. Child components can render different content or styles depending on the values received via props.

Reactivity: When data passed through props changes in the parent component, React automatically updates the child component to reflect the changes. This enables a reactive and synchronized UI.

Practical Scenario:

Let's consider a practical scenario where you would use Props to pass data between components. Imagine you are building a simple "Task List" application with React. You have two main components: `**TaskList**` and `**TaskItem**`.

TaskList Component: This component represents a list of tasks. It maintains an array of tasks and renders each task using the **TaskItem** component.

```jsx
import React from 'react';
import TaskItem from './TaskItem';

function TaskList() {
    const tasks = [
        { id: 1, text: 'Complete homework' },
        { id: 2, text: 'Buy groceries' },
        { id: 3, text: 'Go for a run' },
    ];

    return (
        <div>
            <h2>Task List</h2>
            <ul>
                {tasks.map((task) => (
                    <TaskItem key={task.id} text={task.text} />
                ))}
            </ul>
        </div>
    );
}

export default TaskList;
```

TaskItem Component: This component represents an individual task. It receives the task's text via props and renders it.

```jsx
import React from 'react';

function TaskItem(props) {
    return <li>{props.text}</li>;
}

export default TaskItem;
```

In this scenario:

The **TaskList** component maintains an array of tasks and maps over them to create individual **TaskItem** components.

Each **TaskItem** component receives the text property via props, which is the task description.

By passing the task data through props, we can easily customize and render each task item dynamically.

If you add or modify tasks in the **TaskList** component's state, React will automatically update the UI in response.

This demonstrates the practical use of props in React to pass data from a parent component (**TaskList**) to child components (**TaskItem**) and create a dynamic and reactive user interface.

5. **Describe the role of Class components in React. How do they differ from functional components?**

Class components and functional components are two different ways of creating React components, each with its own syntax and capabilities. As of my last knowledge update in September 2021, I'll describe the role of class components in React and highlight their differences from functional components.

Role of Class Components in React:

Class components were the traditional way of defining components in React before the introduction of functional components and hooks. Here are the primary roles and characteristics of class components:

1. **State Management**: Class components can have state using the this.state property. State allows components to store and manage data that can change over time. State changes trigger re-renders, updating the user interface.
2. **Lifecycle Methods**: Class components can utilize lifecycle methods such as componentDidMount, componentDidUpdate, and componentWillUnmount to perform tasks like fetching data, managing subscriptions, and responding to component updates.
3. **Complex Logic**: Class components are suitable for handling complex logic within the component. They allow you to encapsulate both the UI and the logic in a single class.

Differences from Functional Components:

Functional components have gained popularity in React due to their simplicity and the introduction of hooks. Here are some key differences between class components and functional components:

Syntax: Class components are defined using JavaScript classes and extend the React.Component class. Functional components are defined as JavaScript functions.

Class Component Example:

```
class MyComponent extends React.Component {
    // ...
}
```

Functional Component Example:

```
function MyComponent() {
    // ...
}
```

1. **State Handling**: In class components, you use this.state to manage component state. In functional components, you use the useState hook to manage state.

Class Component:

```
this.state = { count: 0 };
```

Functional Component:

```
const [count, setCount] = useState(0);
```

2. **Lifecycle Methods**: Class components have lifecycle methods like componentDidMount and componentDidUpdate. Functional components can achieve similar functionality using hooks like useEffect.

Class Component:

```
componentDidMount() {
    // perform actions after component mounts
}
```

Functional Component with useEffect:

```
useEffect(() => {
    // perform actions after component mounts
}, []);
```

3. **Readability and Simplicity**: Functional components are often considered more readable and easier to understand, especially for simpler UI components. They have a more concise syntax.
4. **Hooks**: Functional components are compatible with React hooks, which allow you to manage state, side effects, and context more efficiently. Class components do not support hooks.

4. **Explain how conditional statements and operators can be used in React class components. Provide an example.**

Conditional statements and operators can be used in React class components to conditionally render content, apply styles, or manage component behavior based on specific conditions. Here's an explanation of how conditional statements and operators work in React class components, along with an example:

Using Conditional Statements in React Class Components:

Conditional statements like if, else if, and else can be used within the render() method of a React class component to determine what content should be rendered. The basic structure involves evaluating a condition and rendering different JSX based on the result of that condition.

Example of Conditional Rendering:

Let's create a React class component that conditionally renders a greeting message based on the time of day. We'll use the Date object to get the current hour and decide whether to display "Good morning," "Good afternoon," or "Good evening."

```jsx
import React, { Component } from 'react';

class Greeting extends Component {
    render() {
        // Get the current hour
        const currentHour = new Date().getHours();

        let greetingMessage;

        // Use conditional statements to set the greeting message
        if (currentHour >= 5 && currentHour < 12) {
            greetingMessage = "Good morning!";
        } else if (currentHour >= 12 && currentHour < 17) {
            greetingMessage = "Good afternoon!";
        } else {
            greetingMessage = "Good evening!";
        }

        return (
            <div>
                <h1>Greeting</h1>
                <p>{greetingMessage}</p>
            </div>
        );
    }
}

export default Greeting;
```

In this example:

We import **React** and **Component** from the 'react' library.

Inside the **Greeting** class component's **render**() method, we obtain the current hour using new **Date().getHours().**

We declare a variable **greetingMessage** and use conditional statements to set its value based on the current hour. Depending on the time of day, the component will render "Good morning," "Good afternoon," or "Good evening."

Finally, we render the greeting message within a **<p>** element.

Using Conditional Operators in React Class Components:

Conditional operators like the ternary **operator (condition ? trueValue : falseValue)** and logical operators **(&&, ||)** are also commonly used in React class components to achieve conditional rendering or conditional behavior.

Example with Ternary Operator:

Let's modify the previous example to use a ternary operator for conditional rendering:

```jsx
import React, { Component } from 'react';

class Greeting extends Component {
    render() {
        const currentHour = new Date().getHours();
        const greetingMessage =
            currentHour >= 5 && currentHour < 12
                ? "Good morning!"
                : currentHour >= 12 && currentHour < 17
                ? "Good afternoon!"
                : "Good evening!";

        return (
            <div>
                <h1>Greeting</h1>
                <p>{greetingMessage}</p>
            </div>
        );
    }
}

export default Greeting;
```

In this version, we use a ternary operator to directly assign the **greetingMessage** based on the time of day.

Both conditional statements and operators are powerful tools for handling dynamic content and behavior in React class components, allowing you to create flexible and responsive user interfaces.

5. **When working with React events, what are Event objects? How can you pass arguments to event handlers?**

In React, event objects are objects that are passed to event handlers. They contain information about the event, such as the type of event, the target element, and the event's properties.

To pass arguments to event handlers, you can use the event keyword. For example, the following code defines an event handler that takes an argument:

```
const MyComponent = () => {
  const handleClick = (event) => {
    console.log(event.target);
  };

  return (
    <button onClick={handleClick}>Click Me!</button>
  );
};
```

In this example, the **handleClick** event handler takes an event object as its argument. The event object contains information about the click event, such as the target element.

Passing Arguments to Event Handlers:

You can pass additional arguments to event handlers in React using several techniques. Here are two common methods:

1. **Function Bind Syntax:**

You can use arrow functions or the .bind() method to pass arguments to an event handler. Here's an example:

```jsx
import React, { Component } from 'react';

class Button extends Component {
    handleClick = (id, event) => {
        // You can access the event object and the additional argument (id)
        console.log('Button clicked with id:', id);
    };

    render() {
        const id = 42; // Some additional data

        return (
            <button onClick={(event) => this.handleClick(id, event)}>
                Click me
            </button>
        );
    }
}

export default Button;
```

In this example, we pass the **id** as an additional argument to the **handleClick** method by using an arrow function in the **onClick** event handler.

2. Custom Data Attributes:

You can also use custom data attributes to attach data to the DOM element and then access that data in the event handler. Here's an example:

```jsx
import React, { Component } from 'react';

class Button extends Component {
    handleClick = (event) => {
        const id = event.target.getAttribute('data-id');
        console.log('Button clicked with id:', id);
    };

    render() {
        const id = 42; // Some additional data

        return (
            <button onClick={this.handleClick} data-id={id}>
                Click me
            </button>
        );
    }
}

export default Button;
```

In this example, we use the **data-id** attribute to store the additional data (id) on the button element. When the button is clicked, we retrieve the **id** from the **event.target** in the event handler.

These methods allow you to pass data along with event objects to your event handlers in React, making it flexible and convenient to handle events in various scenarios.

Unit 3: Forms and Hooks in React.JS

1. What are the steps involved in adding and handling forms in React? Describe the use of `event.target.name` and `event.target.value` in form handling.

Create functional component through the steps of adding and handling forms in a functional component in React, including the use of event.target.name and event.target.value. We'll build a simple form to collect user information.

Step 1: Create a Form Element

Start by creating a functional component and define the JSX for your form.

```jsx
import React, { useState } from 'react';

function MyForm() {
    return (
        <form>
            <label>
                Name:
                <input type="text" name="name" />
            </label>
            <br />
            <label>
                Email:
                <input type="email" name="email" />
            </label>
            <br />
            <button type="submit">Submit</button>
        </form>
    );
}

export default MyForm;
```

Step 2: Define State for Form Data

Use the useState hook to define state variables for your form data. Create a state variable for each form input.

```
import React, { useState } from 'react';

function MyForm() {
    const [formData, setFormData] = useState({
        name: '',
        email: '',
    });

    return (
        <form>
            {/* ... */}
        </form>
    );
}

export default MyForm;
```

Step 3: Handle Input Changes

Create a function to handle changes in form inputs. Use the name attribute of the input field and event.target.value to update the corresponding state variable.

```jsx
import React, { useState } from 'react';

function MyForm() {
    const [formData, setFormData] = useState({
        name: '',
        email: '',
    });

    const handleInputChange = (event) => {
        const { name, value } = event.target;
        setFormData({
            ...formData,
            [name]: value,
        });
    };

    return (
        <form>
            <label>
                Name:
                <input
                    type="text"
                    name="name"
                    value={formData.name}
                    onChange={handleInputChange}
                />
            </label>
            <br />
            <label>
                Email:
                <input
                    type="email"
                    name="email"
                    value={formData.email}
                    onChange={handleInputChange}
                />
            </label>
```

```
            <br />
            <button type="submit">Submit</button>
        </form>
    );
}

export default MyForm;
```

In the **handleInputChange** function:

We use **event.target.name** to determine which form input is being changed. The **name** attribute of the input field corresponds to the state variable we want to update.

We use **event.target.value** to obtain the new value entered into the input field.

We use the spread operator **(...formData)** to maintain the previous values of the form data and only update the field that is changing.

Step 4: Handle Form Submission

Create a function to handle form submission. Use the **onSubmit** event of the form element to trigger this function.

```jsx
import React, { useState } from 'react';

function MyForm() {
    const [formData, setFormData] = useState({
        name: '',
        email: '',
    });

    const handleInputChange = (event) => {
        const { name, value } = event.target;
        setFormData({
            ...formData,
            [name]: value,
        });
    };

    const handleSubmit = (event) => {
        event.preventDefault(); // Prevent the default form submission beha
        console.log('Form data submitted:', formData);

        // Add code to send data to a server or perform other actions here
    };

    return (
        <form onSubmit={handleSubmit}>
            {/* ... */}
        </form>
    );
}

export default MyForm;
```

In the **handleSubmit** function:

We prevent the default form submission behavior using **event.preventDefault()** to avoid a page refresh.

We can access the form data from the **formData** state variable, which has been updated as the user interacts with the form.

You can add code to send the data to a server, perform validation, or take any other actions as needed.

These steps demonstrate how to create and handle forms in a functional component in React, using **useState** to manage form data and **event.target.name** and **event.target.value** to efficiently handle changes in form inputs.

2. **In React, what is React Memo, and how can it optimize component rendering? Provide an example.**

Using memo will cause React to skip rendering a component if its props have not changed.

This can improve performance.

Problem

In this example, the Todos component re-renders even when the todos have not changed.

```
import { useState } from "react";
import ReactDOM from "react-dom/client";
import Todos from "./Todos";

const App = () => {
  const [count, setCount] = useState(0);
  const [todos, setTodos] = useState(["todo 1", "todo 2"]);

  const increment = () => {
    setCount((c) => c + 1);
  };

  return (
    <>
      <Todos todos={todos} />
      <hr />
      <div>
        Count: {count}
        <button onClick={increment}>+</button>
      </div>
    </>
  );
};

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<App />);
```

Todos.js :

```
const Todos = ({ todos }) => {
  console.log("child render");
  return (
    <>
      <h2>My Todos</h2>
      {todos.map((todo, index) => {
        return <p key={index}>{todo}</p>;
      })}
    </>
  );
};

export default Todos;
```

When you click the increment button, the Todos component re-renders.

If this component was complex, it could cause performance issues.

Solution

To fix this, we can use memo.

Use memo to keep the Todos component from needlessly re-rendering.

Wrap the Todos component export in memo:

Todos.js :

```
import { memo } from "react";

const Todos = ({ todos }) => {
  console.log("child render");
  return (
    <>
      <h2>My Todos</h2>
      {todos.map((todo, index) => {
        return <p key={index}>{todo}</p>;
      })}
    </>
  );
};

export default memo(Todos);
```

Now the Todos component only re-renders when the todos that are passed to it through props are updated.

3. **Discuss the usage of TextArea and Drop-down lists (SELECT) components in React forms. How do you control their values and handle user input?**

In React forms, <**textarea**> and <**select**> (dropdown lists) components are used to collect user input for longer text entries and to provide a list of options, respectively. You can control their values and handle user input by utilizing React's controlled component approach.

1. <textarea> Component:

The <**textarea**> component is used for multiline text input fields, such as comments or descriptions. To control its value and handle user input, follow these steps:

Create a state variable to store the value of the <**textarea**>.

Set the value attribute of the <**textarea**> to the state variable.

Create an event handler function to update the state variable when the user types into the <**textarea**>.

Here's an example:

```jsx
import React, { useState } from 'react';

function TextAreaExample() {
    const [text, setText] = useState('');

    const handleTextChange = (event) => {
        setText(event.target.value);
    };

    return (
        <div>
            <textarea
                value={text}
                onChange={handleTextChange}
                rows="4"
                cols="50"
                placeholder="Enter your text here"
            />
            <p>You entered: {text}</p>
        </div>
    );
}

export default TextAreaExample;
```

In this example, the value of the <**textarea**> is controlled by the text state variable. The **handleTextChange** function updates the text state whenever the user types in the <**textarea**>.

2. <select> (Dropdown List) Component:

The <**select**> component is used to create a dropdown list of options. To control its value and handle user input, you can follow these steps:

Create a state variable to store the selected option's value.

Set the value attribute of each <**option**> element to the corresponding option value.

Set the value attribute of the <**select**> element to the state variable.

Create an event handler function to update the state variable when the user selects an option.

Here's an example:

```
import React, { useState } from 'react';

function SelectExample() {
    const [selectedOption, setSelectedOption] = useState('option1');

    const handleOptionChange = (event) => {
        setSelectedOption(event.target.value);
    };

    return (
        <div>
            <select value={selectedOption} onChange={handleOptionChange}>
                <option value="option1">Option 1</option>
                <option value="option2">Option 2</option>
                <option value="option3">Option 3</option>
            </select>
            <p>Selected option: {selectedOption}</p>
        </div>
    );
}

export default SelectExample;
```

In this example, the selected option in the <**select**> is controlled by the **selectedOption** state variable. The **handleOptionChange** function updates the **selectedOption** state whenever the user selects a different option.

Using these controlled component approaches for <**textarea**> and <**select**> elements in React forms ensures that the component's value is always in sync with the state, allowing you to manage user input effectively and provide a responsive user experience.

4. **Explain the concepts and advantages of React Hooks. How do they simplify state management in functional components?**

React Hooks are a set of functions introduced in React 16.8 that allow functional components to manage state, side effects, and other React features previously available only in class components. Hooks were introduced to simplify and enhance the development of functional components by providing a more concise and intuitive way to manage component logic. Here's an explanation of the concepts and advantages of React Hooks:

Concepts of React Hooks:

1. **State Hook (useState):** The useState hook allows functional components to manage local state. It returns a state variable and a function to update that variable. This enables functional components to have their own component-level state.

```
const [count, setCount] = useState(0);
```

2. **Effect Hook (useEffect):** The useEffect hook handles side effects, such as data fetching, DOM manipulation, and subscriptions. It is used to perform actions after the component renders or when certain dependencies change.

```
useEffect(() => {
    // Perform side effects here
}, [dependencies]);
```

3. **Context Hook (useContext):** The useContext hook allows functional components to access context values provided by a Context.Provider higher in the component tree. It simplifies passing data to deeply nested components.

```
const value = useContext(MyContext);
```

4. **Ref Hook (useRef):** The useRef hook creates a mutable ref object that can hold a reference to a DOM element or any value that persists across renders. It is useful for accessing and interacting with DOM elements directly.

```
const myRef = useRef(initialValue);
```

5. **Reducer Hook (useReducer):** The useReducer hook is an alternative to useState for managing complex state logic. It allows you to manage state updates through a reducer function, similar to how Redux works.

```
const [state, dispatch] = useReducer(reducer, initialArg, init);
```

6. Custom Hooks: Developers can create their own custom hooks to encapsulate reusable logic. Custom hooks follow the naming convention use..., and they can be shared across components.

Advantages of React Hooks:

- **Simplified Logic**: Hooks allow you to reuse stateful logic without changing the component hierarchy. This reduces the complexity of components and promotes code reuse.
- **Easier to Read and Test**: Functional components using hooks tend to be more concise and easier to read compared to class components with lifecycle methods. This makes them more testable and maintainable.
- **Reduced Boilerplate:** Hooks eliminate the need for writing class components, constructor methods, and binding functions, reducing boilerplate code.
- **Better Performance:** Hooks enable more optimizations in React's rendering process, potentially leading to better performance in certain scenarios.
- **Improved Reusability**: Custom hooks promote the reuse of logic across components and even across projects, improving code modularity.
- **Simplified Side Effects**: The useEffect hook simplifies handling side effects by consolidating them in one place, making it easier to manage asynchronous operations.
- **Easier Learning Curve**: Hooks provide a more consistent and intuitive way of working with component logic, which can make it easier for developers, especially those new to React, to learn and work with.

In summary, React Hooks simplify state management, side effects, and other component logic in functional components. They provide a more elegant and maintainable approach to building React applications, reducing the need for class components and making code more readable and testable. As a result, Hooks have become an essential tool in modern React development.

7. **Provide examples and use cases for the following React Hooks: `useState`, `useEffect`, and `useContext`.**

1. useState Hook:

The useState hook allows functional components to manage local state. Here's an example:

```jsx
import React, { useState } from 'react';

function Counter() {
    // Initialize state with a count value of 0
    const [count, setCount] = useState(0);

    return (
        <div>
            <p>Count: {count}</p>
            <button onClick={() => setCount(count + 1)}>Increment</button>
        </div>
    );
}

export default Counter;
```

Use Case: Managing local component state, such as a form input's value, toggling UI elements, or keeping track of user interactions.

2. useEffect Hook:

The useEffect hook is used for handling side effects and performing actions after a component renders or when certain dependencies change. Here's an example of fetching data from an API:

```jsx
import React, { useState, useEffect } from 'react';

function DataFetcher() {
    const [data, setData] = useState([]);

    useEffect(() => {
        // Fetch data from an API when the component mounts
        fetch('https://api.example.com/data')
            .then((response) => response.json())
            .then((data) => setData(data));
    }, []); // Empty dependency array to run the effect only once

    return (
        <div>
            <h2>Fetched Data:</h2>
            <ul>
                {data.map((item) => (
                    <li key={item.id}>{item.name}</li>
                ))}
            </ul>
        </div>
    );
}

export default DataFetcher;
```

Use Case: Handling data fetching, DOM manipulation, subscriptions, or any other side effects in a component.

3. useContext Hook:

The useContext hook allows functional components to access context values provided by a Context.Provider higher in the component tree. Here's an example:

```jsx
import React, { useContext } from 'react';

// Create a context
const ThemeContext = React.createContext('light');

function ThemeSwitcher() {
    // Access the theme value from context
    const theme = useContext(ThemeContext);

    return (
        <div>
            <p>Current Theme: {theme}</p>
        </div>
    );
}

function App() {
    return (
        <ThemeContext.Provider value="dark">
            <div>
                <h1>Theme Switcher App</h1>
                <ThemeSwitcher />
            </div>
        </ThemeContext.Provider>
    );
}

export default App;
```

Use Case: Accessing global configuration, themes, user authentication, or any shared data without prop drilling.

These examples demonstrate how to use the useState, useEffect, and useContext hooks in React for managing component state, handling side effects, and accessing context data. Each hook simplifies specific aspects of component development and contributes to building more efficient and maintainable React applications.

8. **What is the purpose of `useRef` in React? Give an example of when you might use `useRef`.**

Use useRef in React to create a simple counter that persists its value between renders without causing re-renders:

```
import React, { useRef, useState } from 'react';

function Counter() {
    const counterRef = useRef(0); // Initialize a ref with an initial value
    const [count, setCount] = useState(0); // State for rendering

    const handleIncrement = () => {
        counterRef.current++; // Update the ref value
        setCount(counterRef.current); // Update the state with the ref valu
    };

    return (
        <div>
            <h2>Counter</h2>
            <p>Count: {count}</p>
            <button onClick={handleIncrement}>Increment</button>
        </div>
    );
}

export default Counter;
```

In this example:

We create a ref called **counterRef** and initialize it with a value of 0.

When the "Increment" button is clicked, **handleIncrement** is called. Inside this function, we increment the value of **counterRef.current**

We then update the component's state with **setCount(counterRef.current).** This update causes a re-render, but since we're using a ref to hold the value, it doesn't create an infinite loop of re-renders.

This example demonstrates how you can use **useRef** to preserve a value between renders without causing unnecessary re-renders of the component. It's a useful technique for managing values that don't need to be part of the component's state but still need to persist across renders.

9. **Describe the use of `useReducer`, `useCallback`, and `useMemo` hooks in React. How do they enhance the performance and maintainability of your components?**

useReducer, **useCallback**, and **useMemo** are React hooks that enhance the performance and maintainability of components in different ways. Let's explore each of these hooks and their use cases:

1. useReducer Hook:

The **useReducer** hook is used for managing complex state logic that involves multiple actions and transitions. It is particularly beneficial when you need to manage state transitions in a predictable and controlled manner. **useReducer** is similar to **Redux** and encourages a unidirectional data flow.

Use Cases:

Managing state that has complex interactions and transitions.

Handling state changes that depend on previous state values.

Reducing the complexity of component state management.

Example:

```
import React, { useReducer } from 'react';

// Reducer function
const counterReducer = (state, action) => {
    switch (action.type) {
        case 'INCREMENT':
            return { count: state.count + 1 };
        case 'DECREMENT':
            return { count: state.count - 1 };
        default:
            return state;
    }
};
```

```
function Counter() {
    const [state, dispatch] = useReducer(counterReducer, { count: 0 });

    return (
        <div>
            <p>Count: {state.count}</p>
            <button onClick={() => dispatch({ type: 'INCREMENT' })}>Incremer
            <button onClick={() => dispatch({ type: 'DECREMENT' })}>Decremer
        </div>
    );
}

export default Counter;
```

2. useCallback Hook:

The **useCallback** hook is used to memoize functions, preventing unnecessary re-creations of those functions on every render. This is especially helpful when passing functions as props to child components or when functions have dependencies that you want to ensure remain stable.

Use Cases:

Preventing unnecessary re-renders of child components due to function prop changes.

Optimizing performance when using functions in **useEffect** dependencies.

Ensuring that callback functions maintain consistent references.

Example:

```jsx
import React, { useState, useCallback } from 'react';

function Parent() {
    const [count, setCount] = useState(0);

    // Memoized callback function
    const increment = useCallback(() => {
        setCount(count + 1);
    }, [count]);

    return (
        <div>
            <p>Count: {count}</p>
            <Child onIncrement={increment} />
        </div>
    );
}
function Child({ onIncrement }) {
    return (
        <div>
            <button onClick={onIncrement}>Increment</button>
        </div>
    );
}

export default Parent;
```

3. useMemo Hook:

The **useMemo** hook is used to memoize the result of an expensive computation, so it's only recalculated when its dependencies change. This can be useful for optimizing expensive calculations or preventing unnecessary computations in rendering.

Use Cases:

Memoizing the result of a computationally expensive operation.

Avoiding re-computation of values that depend on certain props or state.

Enhancing rendering performance by minimizing unnecessary calculations.

Example:

```
import React, { useState, useMemo } from 'react';

function ExpensiveCalculation({ input }) {
    // Memoized result of the expensive calculation
    const result = useMemo(() => {
        // Simulate a computationally expensive operation
        let sum = 0;
        for (let i = 1; i <= input; i++) {
            sum += i;
        }
        return sum;
    }, [input]);

    return <p>Result: {result}</p>;
}
```

```
function App() {
    const [input, setInput] = useState(10);

    return (
        <div>
            <input
                type="number"
                value={input}
                onChange={(e) => setInput(Number(e.target.value))}
            />
            <ExpensiveCalculation input={input} />
        </div>
    );
}

export default App;
```

In summary, **useReducer**, **useCallback**, and **useMemo** are React hooks that address specific use cases to enhance the performance and maintainability of your components. They help you manage state transitions, prevent unnecessary re-renders, and optimize expensive computations, respectively. Choosing the right hook for the job can lead to more efficient and maintainable React applications.

10. **What is a custom hook in React? How can you build a custom hook, and what are the advantages of using them in your application?**

A custom hook in React is a JavaScript function that encapsulates a specific piece of reusable logic. Custom hooks allow you to extract and share stateful logic or side-effect logic across different components in your application. They follow a naming convention of starting with "use" to make it clear that they are hooks.

Build a Hook

In the following code, we are fetching data in our Home component and displaying it.

We will use the **JSONPlaceholder** (https://jsonplaceholder.typicode.com/) service to fetch fake data. This service is great for testing applications when there is no existing data.

To learn more, check out the JavaScript Fetch API section.

Use the JSONPlaceholder service to fetch fake "todo" items and display the titles on the page:

useFetch.js :

```
import { useState, useEffect } from "react";

const useFetch = (url) => {
  const [data, setData] = useState(null);

  useEffect(() => {
    fetch(url)
      .then((res) => res.json())
      .then((data) => setData(data));
  }, [url]);

  return [data];
};

export default useFetch;
```

index.js :

```
import ReactDOM from "react-dom/client";
import useFetch from "./useFetch";

const Home = () => {
  const [data] = useFetch("https://jsonplaceholder.typicode.com/todos");

  return (
    <>
      {data &&
        data.map((item) => {
          return <p key={item.id}>{item.title}</p>;
        })}
    </>
  );
};

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<Home />);
```

Example Explained

We have created a new file called useFetch.js containing a function called useFetch which contains all of the logic needed to fetch our data.

We removed the hard-coded URL and replaced it with a url variable that can be passed to the custom Hook.

Lastly, we are returning our data from our Hook.

In index.js, we are importing our useFetch Hook and utilizing it like any other Hook. This is where we pass in the URL to fetch data from.

Now we can reuse this custom Hook in any component to fetch data from any URL.